InnovShop – Dossier de projet

Nicolas Teixeira – Titre professionnel Développeur web et web mobile – Session du 2 novembre 2026

Table des matières

- 1. Présentation du projet
- 2. Besoins fonctionnels et backlog
- 3. Organisation et méthode
- 4. Technologies et architecture
- 5. Conception
- 6. Réalisation
- 7. Sécurité
- 8. Tests et qualité
- 9. Performance
- 10. Déploiement
- 11. Bilan et perspectives
- Annexes

1. Présentation du projet

InnovShop est un site e-commerce de produits innovants, développé avec Symfony 7.4 et mis en ligne sur Alwaysdata, qui permet de parcourir un catalogue, de remplir un panier et de passer commande.

Contexte et objectifs

Ce projet est le projet final de ma formation de développeur web. Il consiste à concevoir, réaliser, tester et déployer un site e-commerce complet, puis à le présenter devant un jury.

Les objectifs que je me suis fixés :

- couvrir l'intégralité du parcours d'achat : catalogue, panier, commande, e-mails de suivi ;
- proposer un espace d'administration pour gérer la boutique ;
- protéger les données et les accès (rôles, autorisations, protection CSRF) ;
- garantir la qualité par des tests automatisés et un style de code vérifié ;
- mettre le site en ligne et documenter son installation.

Périmètre

Le site s'adresse à cinq profils d'utilisateurs : le visiteur, le client connecté, le vendeur, le testeur et l'administrateur. Deux ensembles de fonctionnalités le distinguent d'une boutique classique :

- **une marketplace** : un client peut demander à devenir vendeur, l'administrateur valide la demande, puis le vendeur propose ses produits, qui sont modérés avant publication ;
- **une catégorie « Prototypes »** : des produits encore en phase de prototype, que seuls les testeurs validés peuvent commander, avec une page « Devenir testeur » pour candidater.

Liens

Élément	Lien
Site en ligne	ntx.alwaysdata.net/innovshop
Dépôt GitHub	github.com/nicolasteixeira16-cell/innovshop
Kanban Trello (public)	Tableau Trello public

2. Besoins fonctionnels et backlog

Le site couvre cinq profils d'utilisateurs et l'ensemble du parcours d'achat ; les fonctionnalités ci-dessous sont toutes réalisées, testées et en ligne.

Fonctionnalités réalisées

Profil	Fonctionnalités
Visiteur	Page d'accueil (produits à la une, prototypes, nouveautés) ; catalogue avec recherche par mot-clé, filtre par catégorie et tri ; fiche produit avec ses options ; panier (ajout, quantités, suppression, vidage) ; inscription et connexion
Client connecté	Commande en 4 étapes (récapitulatif, livraison, paiement, confirmation) ; e-mail de confirmation et e-mail à chaque changement de statut ; historique « Mes commandes » et facture PDF ; gestion du profil, de l'adresse et du mot de passe
Administrateur	Gestion des catégories, produits, utilisateurs et commandes dans EasyAdmin ; changement de statut des commandes ; consultation des candidatures de testeurs

Marketplace et prototypes

Deux ensembles de fonctionnalités enrichissent la boutique classique.

Ajout	Fonctionnement
Marketplace	Un client demande à devenir vendeur ; l'administrateur lui attribue le rôle ; le vendeur crée, modifie et supprime ses propres produits dans un espace dédié ; tout produit créé ou modifié passe « en attente » et n'est publié qu'après validation de l'administrateur

Ajout	Fonctionnement
Prototypes et testeurs	Une catégorie « Prototypes » regroupe des produits en phase de test ; un visiteur candidate via la page « Devenir testeur » ; l'administrateur valide en attribuant le rôle Testeur ; seuls les testeurs validés (et les administrateurs) peuvent commander un prototype

Backlog

Les user stories détaillées, avec leur priorité et leur avancement, sont regroupées dans le tableau Kanban public (voir la section 3) et reprises en annexe.

3. Organisation et méthode

Le projet a été mené en solo, par étapes successives, avec un tableau Kanban public pour suivre l'avancement et un dépôt Git pour garder l'historique de chaque évolution.

Suivi du travail

Les tâches (user stories) sont gérées dans un tableau Kanban Trello public : chaque carte avance de la colonne des tâches à faire vers celle des tâches terminées. La capture du tableau figure en annexe 2.

Chaque évolution du site est validée par la suite de tests automatisés avant d'être envoyée sur GitHub, puis déployée en production.

Environnement de travail

Besoin	Outil
Éditeur de code	Visual Studio Code
Serveur de développement	Symfony CLI
Base de données locale	MariaDB (XAMPP)
E-mails en développement	Mailpit
Dépendances PHP	Composer
Versionnage	Git et GitHub
Qualité du code	PHPUnit et PHP-CS-Fixer
Hébergement	Alwaysdata (SSH, Git, MariaDB)

4. Technologies et architecture

InnovShop repose sur Symfony 7.4 et une base MariaDB, avec une architecture MVC qui sépare les contrôleurs, la logique métier (services), l'accès aux données (repositories Doctrine) et l'affichage (templates Twig).

Technologies retenues

Domaine	Technologie	Rôle dans le projet
Langage	PHP 8.2 ou supérieur (8.4 en production)	Langage du framework
Framework	Symfony 7.4	Routage, sécurité, formulaires, validation, e-mails
Base de données	MariaDB, Doctrine ORM et migrations	Stockage des données, évolution du schéma versionnée
Vues	Twig	Gabarits et héritage de mise en page
Interface	Bootstrap 5.3 et Turbo	Mise en page responsive, navigation fluide
Ressources	AssetMapper	Gestion et compilation des styles, images et scripts
Administration	EasyAdmin 5	Back-office de la boutique
E-mails	Symfony Mailer	Confirmation de commande et suivi de statut
Tests	PHPUnit, DAMA Doctrine Test Bundle	Tests unitaires et fonctionnels sur base isolée
Qualité	PHP-CS-Fixer	Style de code uniforme (règles Symfony)

Organisation du code

Dossier	Contenu
`src/Controller`	Contrôleurs du site, du compte, de la commande, de l'espace vendeur et de l'administration
`src/Entity`	Entités Doctrine : catégories, produits, options, utilisateurs, adresses, commandes, candidatures
`src/Repository`	Requêtes sur les données
`src/Service`	Logique métier : panier, création de commande, envoi des e-mails
`src/Security/Voter`	Règles d'autorisation (commandes, produits)
`src/Form`	Formulaires et leurs contraintes de validation
`templates`	Vues Twig
`tests`	Tests PHPUnit
`migrations`	Migrations de base de données

Choix de conception

- **Panier en session** : le panier est stocké dans la session et non en base, ce qui évite une requête SQL à chaque page ; le nombre d'articles de la barre de navigation est calculé

sans interroger la base.

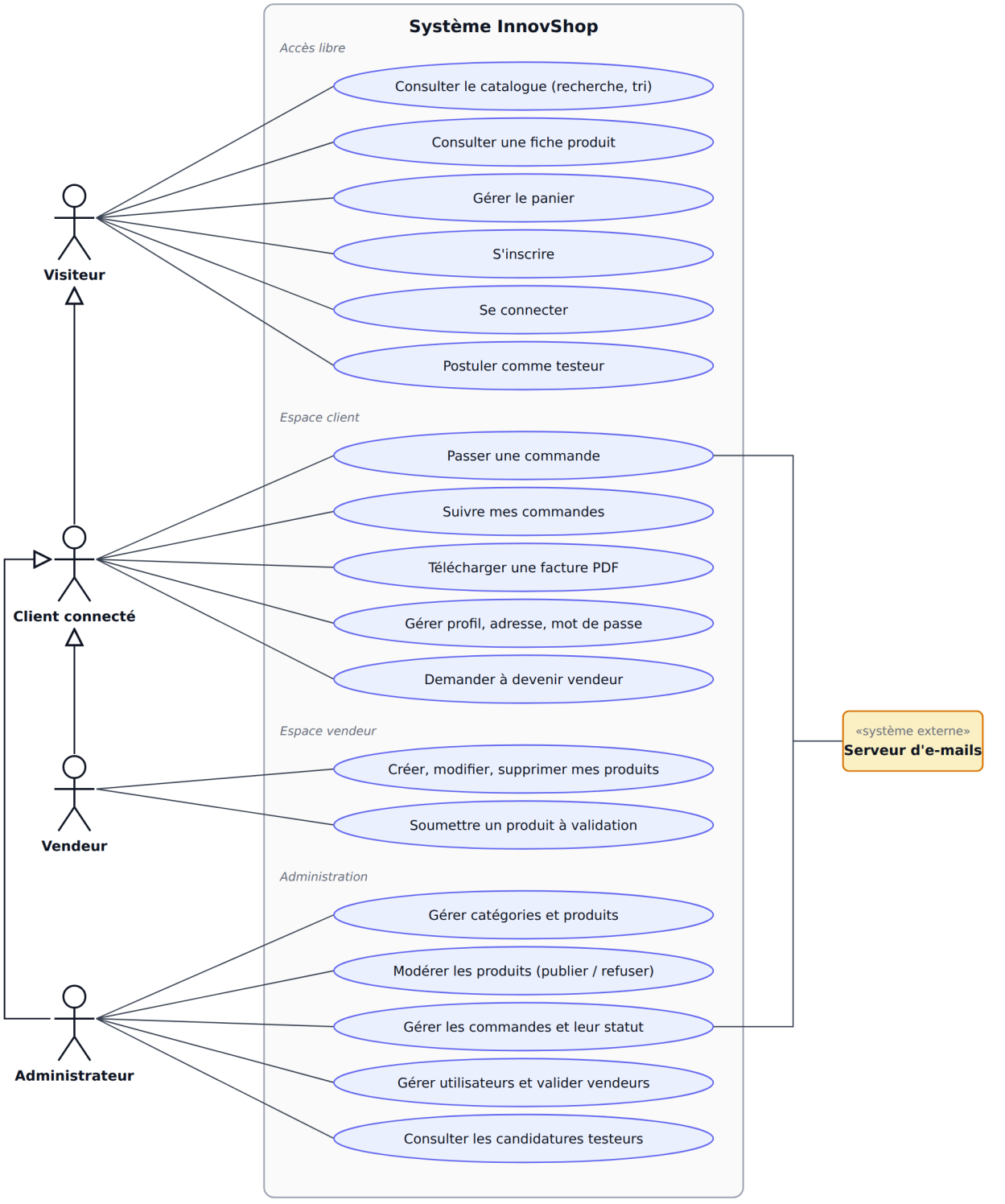
- **Montants en centimes** : les calculs d'argent se font sur des entiers pour éviter les erreurs d'arrondi.
- **Historique figé des commandes** : une ligne de commande conserve le nom et le prix du produit au moment de l'achat, ce qui préserve l'historique même si le produit est modifié ou supprimé.
- **Règles d'accès centralisées** : les autorisations qui dépendent d'un objet (qui peut voir une commande, modifier un produit, commander un prototype) sont regroupées dans des Voters, plutôt qu'éparpillées dans les contrôleurs.

5. Conception

La conception d'InnovShop s'appuie sur cinq diagrammes UML qui décrivent qui fait quoi, comment les données sont organisées et comment se déroulent les deux parcours les plus importants : la commande et la validation d'un produit de vendeur.

Cas d'utilisation

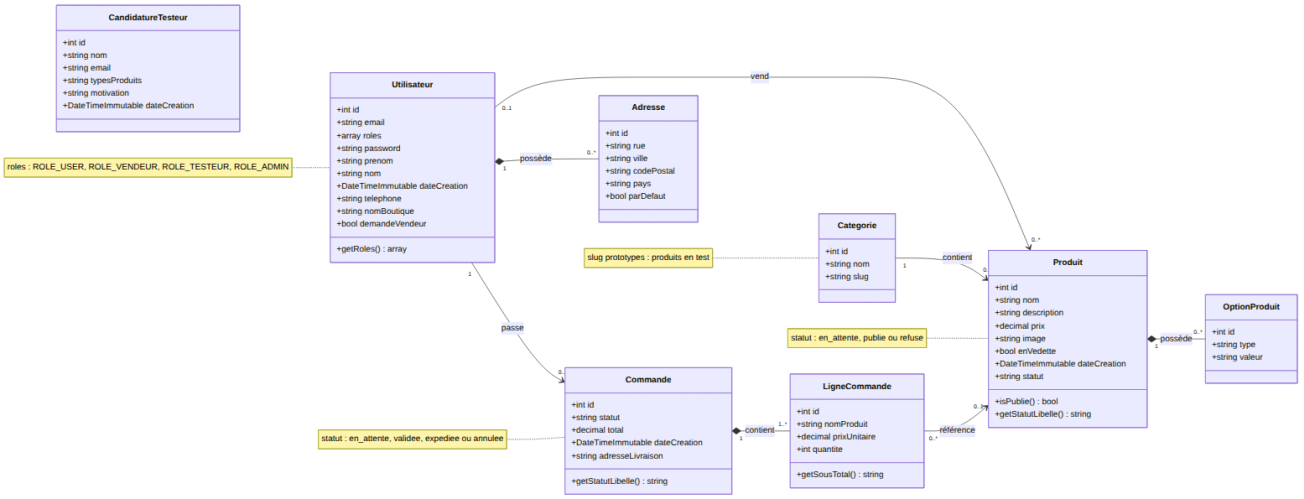
Quatre acteurs interviennent : le visiteur, le client connecté, le vendeur et l'administrateur. Chacun hérite des droits du précédent (un vendeur est aussi un client, un client est aussi un visiteur), et le serveur d'e-mails est un acteur externe qui reçoit les messages de commande. Le testeur est un client connecté (le visiteur dépose sa candidature, l'administrateur lui attribue le rôle) qui peut en plus commander les produits de la catégorie « Prototypes » ; il n'apparaît donc pas comme un acteur distinct dans ce diagramme.



Flèche à triangle creux : généralisation (le Vendeur et l'Administrateur sont aussi des Clients ; le Client est aussi un Visiteur).

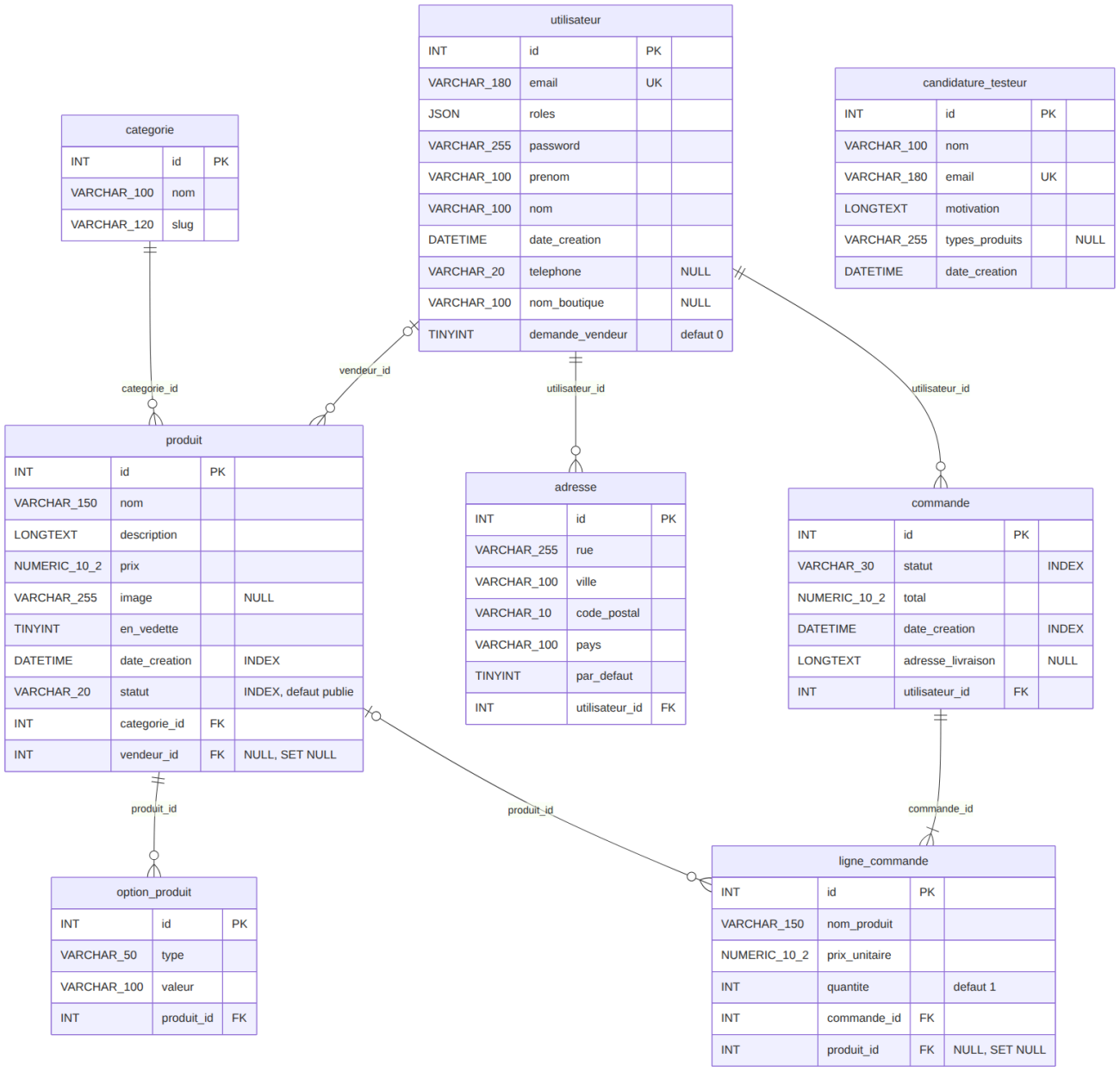
Diagramme de classes

Le modèle comporte huit classes. Un produit appartient à une catégorie et peut avoir un vendeur (un utilisateur). Une commande contient des lignes de commande, qui référencent un produit tout en conservant son nom et son prix d'achat.



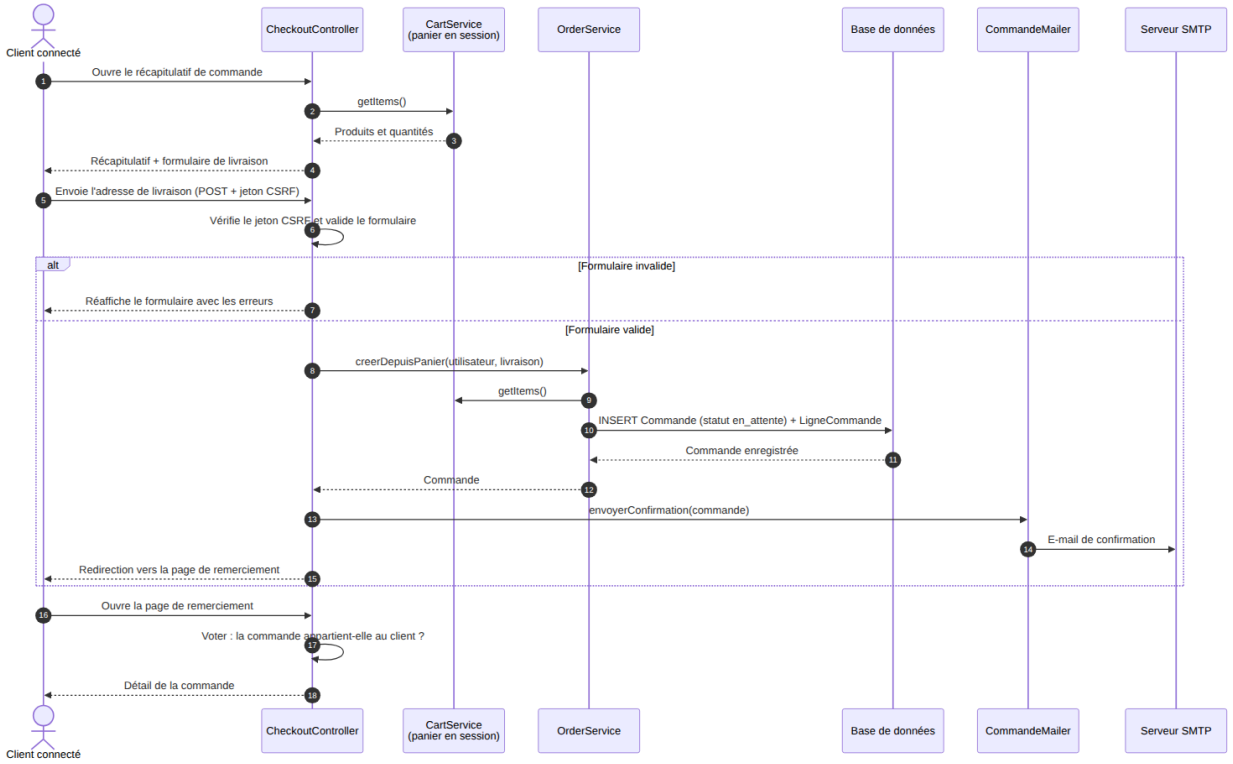
Modèle physique de données

Le schéma relationnel est celui généré par Doctrine sur MariaDB. Les suppressions sont protégées : supprimer un produit conserve ses lignes de commande (la clé étrangère passe à vide), et supprimer un vendeur conserve ses produits. L'adresse e-mail d'un utilisateur est unique, et les colonnes les plus filtrées (statut et date des produits, statut et date des commandes) sont indexées.



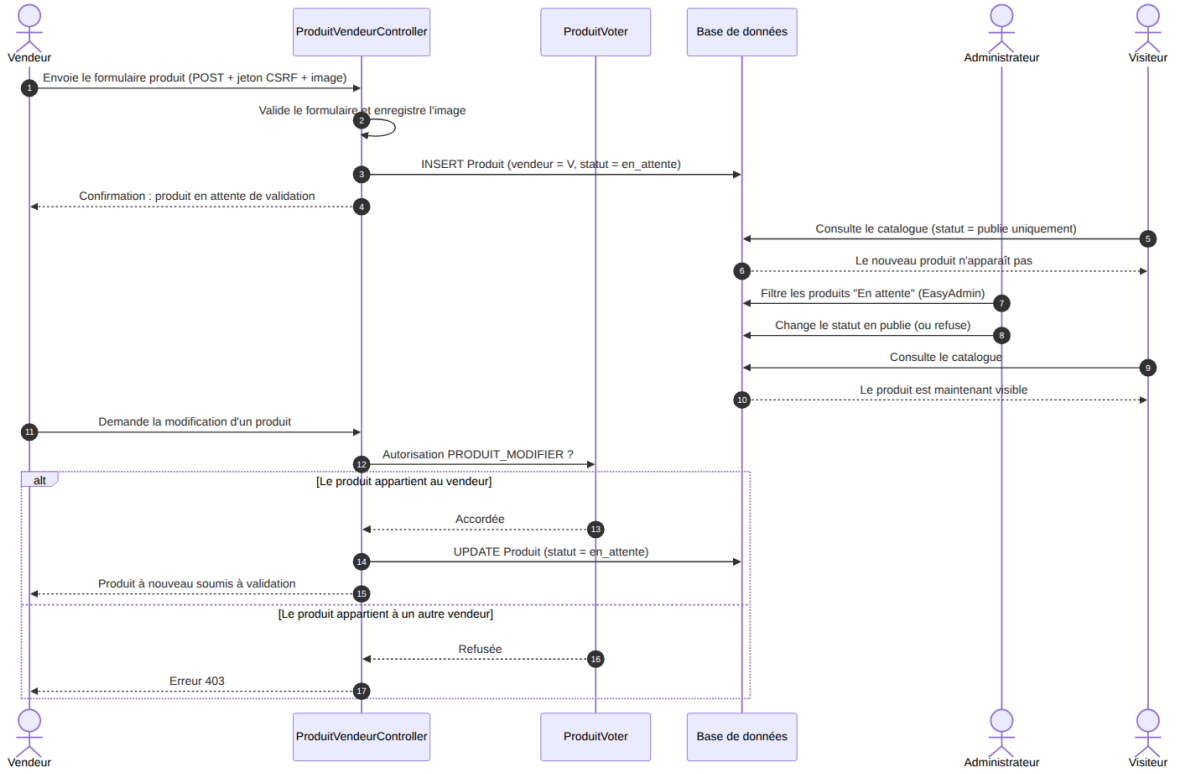
Séquence : passer une commande

Le client vérifie son panier, saisit son adresse de livraison, puis valide : la commande et ses lignes sont enregistrées, un e-mail de confirmation est envoyé, et la page de remerciement n'est accessible qu'au propriétaire de la commande.



Séquence : proposer un produit (marketplace)

Un produit soumis par un vendeur est enregistré « en attente » et reste invisible du public ; il apparaît dans le catalogue après validation par l'administrateur. Un vendeur ne peut modifier que ses propres produits, ce que vérifie un Voter.



6. Réalisation

Le site est réalisé en modules indépendants, chacun avec ses contrôleurs, ses vues et ses tests ; le tableau ci-dessous résume ce que fait chaque module et le point technique qui le distingue.

Module	Ce qu'il fait	Point technique
Catalogue	Liste les produits publiés, avec recherche par mot-clé, filtre par catégorie et tri par prix, nom ou date	Une seule requête Doctrine paramétrée, qui charge aussi la catégorie pour éviter des requêtes en cascade
Fiche produit	Affiche l'image, la description, le prix et les options (caractéristiques) du produit	Description saisie dans un éditeur riche et assainie à l'affichage
Panier	Ajoute, diminue, retire et vide, avec mise à jour du compteur sans recharger la page	Panier en session, quantité limitée à 99 par produit, protection CSRF sur chaque action, réponse JSON pour les appels AJAX
Commande	Tunnel en 4 étapes : récapitulatif, livraison, paiement par carte (simulé : contrôle du numéro, de l'expiration et du cryptogramme, aucune donnée de carte conservée), confirmation ; enregistre la commande puis affiche une page de remerciement	Adresse de livraison pré-remplie avec l'adresse par défaut ; page de remerciement réservée au propriétaire par un Voter
Compte client	Profil, adresse, mot de passe, historique des commandes et facture PDF	Accès restreint à l'utilisateur connecté par les règles d'accès de sécurité
E-mails	Confirmation de commande et message à chaque changement de statut	Service dédié ; la commande reste enregistrée même si l'envoi échoue, et le client en est averti
Administration	Gestion des catégories, produits (avec envoi d'image), utilisateurs, commandes et candidatures	EasyAdmin ; suppression d'un utilisateur désactivée pour conserver l'historique ; filtre par statut sur les produits
Marketplace	Demande de vendeur, espace vendeur, modération des produits	Rôles `ROLE_VENDEUR` avec hiérarchie, statut du produit (en attente, publié, refusé), propriété vérifiée par un Voter
Prototypes et testeurs	Catégorie « Prototypes », candidature de testeur, commande réservée aux testeurs validés	Rôle `ROLE_TESTEUR` ; règle « commander » appliquée dans les vues, à l'ajout au panier et à la validation de la commande

Identité visuelle

Pour que le site ait une identité propre plutôt que l'aspect d'un gabarit Bootstrap par défaut, j'ai défini un thème regroupé dans un seul fichier de styles, `theme.css`.

Élément	Choix	Raison
Palette	Bleu nuit `#0f1b2d`, turquoise `#14b8a6`, orange `#ff8a3d`	Univers technologique (bleu nuit), couleur principale pour les actions (turquoise), accent pour les prototypes et les appels à l'action (orange)
Typographie	Space Grotesk pour les titres, Inter pour le texte	Un titre expressif, un texte très lisible
Logo et bandeau	Cube isométrique créé pour le projet, décliné en logo et en image d'accueil	Une image de « prototype », sans visuel emprunté
Cartes produit	Ombre légère, relief et zoom de l'image au survol	Retour visuel pour inciter au clic

Les couleurs de Bootstrap sont redéfinies par des variables CSS, ce qui fait suivre toute l'interface (boutons, liens, formulaires) sans réécrire chaque composant.

7. Sécurité

La sécurité est traitée à chaque niveau du site : qui peut se connecter, ce que chaque rôle a le droit de faire, et comment les données saisies sont contrôlées avant d'être enregistrées ou affichées.

Risque	Mesure mise en place
Vol de mot de passe	Mots de passe hachés avec bcrypt, jamais stockés ni affichés en clair ; l'inscription impose 8 caractères minimum et une force de mot de passe moyenne
Accès à une zone interdite	Quatre rôles (`ROLE_USER`, `ROLE_VENDEUR`, `ROLE_TESTEUR`, `ROLE_ADMIN`) avec une hiérarchie ; des règles d'accès par zone du site (administration, commande, compte, espace vendeur)
Accès à l'objet d'un autre utilisateur	Des Voters vérifient que la commande consultée appartient bien au client et que le produit modifié appartient bien au vendeur
Commande d'un prototype sans autorisation	Une règle unique (`PRODUIT_COMMANDER`) appliquée dans les vues, à l'ajout au panier et à la validation de la commande
Requête forgée depuis un autre site (CSRF)	Jeton CSRF sur la connexion, la déconnexion, les formulaires et toutes les actions du panier et de la commande
Injection de HTML ou de script (XSS)	Échappement automatique de Twig ; descriptions de produits assainies avec `sanitize_html` ; rejet des caractères `<` et `>` dans les champs de texte libre

Risque	Mesure mise en place
Données incorrectes ou en double	Contraintes de validation sur les formulaires et les entités ; e-mail unique pour les comptes et les candidatures
Fuite de configuration	Secrets uniquement dans un fichier `.env.local` ignoré par Git ; en production, mode `prod` et débogage désactivé
Produit non validé visible du public	Seuls les produits publiés apparaissent au catalogue, à l'accueil et au panier ; une fiche en attente renvoie une erreur 404 sauf pour son vendeur et l'administrateur

8. Tests et qualité

La suite automatisée compte 60 tests et 125 assertions, tous au vert, et le code est vérifié par un outil de style avant chaque envoi sur GitHub.

Tests automatisés

Domaine testé	Ce qui est vérifié
Catalogue	Recherche, filtre par catégorie et tri ; seuls les produits publiés apparaissent
Accès et sécurité	Redirection des visiteurs vers la connexion, refus (403) des rôles insuffisants, refus d'une action du panier sans jeton CSRF
Marketplace	Visibilité des produits en attente, accès à l'espace vendeur, propriété des produits (un vendeur ne modifie ni ne supprime le produit d'un autre), demande de vendeur
Prototypes	Un client sans droit ne peut pas ajouter un prototype ; un testeur et un administrateur le peuvent ; la fiche produit adapte son bouton ; la règle du Voter est testée seule, sans base de données

Les tests fonctionnels simulent un navigateur (requêtes HTTP, connexion d'un utilisateur, soumission de formulaires). Chaque test s'exécute dans une transaction annulée à la fin, grâce à DAMA Doctrine Test Bundle : la base de test reste propre et les tests n'interfèrent pas entre eux.

Qualité du code

PHP-CS-Fixer applique les règles de style de Symfony ; il est lancé avant chaque commit et corrige automatiquement les écarts.

Données de démonstration

Des fixtures Doctrine créent en une commande les catégories, les produits (avec leurs images) et quatre comptes de démonstration (administrateur, client, vendeur, testeur). Elles ne sont installées qu'en développement : en production, elles ne peuvent pas être exécutées. Une commande dédiée, `app:creer-admin`, crée un administrateur avec les identifiants de son choix.

9. Performance

Les performances reposent sur trois leviers : limiter les requêtes à la base de données, mettre en cache les fichiers statiques pendant un an et les compresser avant l'envoi.

Levier	Mise en œuvre	Résultat vérifié en production
Requêtes optimisées	Chargement de la catégorie avec le produit dans une seule requête ; compteur du panier calculé depuis la session, sans SQL	Pas de requête supplémentaire par produit affiché
Index de base de données	Index sur le statut et la date des produits, sur le statut et la date des commandes	Filtres et tris sans parcours complet des tables
Cache navigateur	Fichiers de `/assets/` servis avec `Cache-Control: public, max-age=31536000, immutable` ; le nom de chaque fichier change quand son contenu change	En-tête présent sur le CSS et le JavaScript en ligne
Compression	Compression gzip des pages HTML, CSS, JavaScript, JSON, XML et SVG (`public/.htaccess`)	`Content-Encoding: gzip` présent sur la page d'accueil et sur le JavaScript
Ressources compilées	`asset-map:compile` génère les fichiers publics versionnés à chaque déploiement	Fichiers signés par leur contenu

Les vérifications ont été faites en production avec `curl`, en lisant les en-têtes de réponse d'une vraie requête.

Audit Lighthouse

Le site en production a été audité avec Lighthouse (page d'accueil, mode mobile avec connexion 4G ralentie simulée, le 30 octobre 2026). Les notes obtenues sont de 85 pour la performance, 92 pour l'accessibilité, 100 pour les bonnes pratiques et 83 pour le référencement.

L'audit relève plusieurs points d'amélioration : une balise de description de la page absente, une hiérarchie de titres à revoir, le contraste de certains textes, et du CSS et du JavaScript chargés mais peu utilisés. La vérification du fichier robots.txt échoue car Lighthouse le cherche à la racine du domaine mutualisé d'Alwaysdata, en dehors du dossier du site.

10. Déploiement

Le site est hébergé sur Alwaysdata, à l'adresse ntx.alwaysdata.net/innovshop, et se met à jour en quelques commandes à partir du dépôt GitHub.

Configuration de production

Élément	Choix
Hébergement	Alwaysdata, accès SSH, PHP 8.4 en ligne de commande

Élément	Choix
Code	Cloné depuis GitHub dans `~/www/innovshop` ; le site est servi depuis son dossier `public/`
Base de données	MariaDB 11.4 d'Alwaysdata, avec un utilisateur dédié à l'application
Paramètres sensibles	Fichier `.env.local` présent uniquement sur le serveur : mode `prod`, débogage désactivé, secret de l'application, accès à la base, serveur SMTP
E-mails	Serveur SMTP d'Alwaysdata ; confirmation de commande et changement de statut vérifiés par réception
Dépendances	Installées sans les outils de développement (`--no-dev`) : ni tests ni fixtures en production

Procédure de mise à jour

1. Envoyer les modifications sur GitHub (`git push`).
2. Se connecter en SSH et récupérer le code : `git pull`.
3. Installer les dépendances de production : `composer install --no-dev --optimize-autoloader`.
4. Appliquer les migrations de base de données : `php bin/console doctrine:migrations:migrate --no-interaction`.
5. Compiler les ressources : `php bin/console asset-map:compile`.
6. Vider le cache : `php bin/console cache:clear`.

Le dépôt GitHub est public et ne contient aucun secret : tous les paramètres sensibles restent sur le serveur.

11. Bilan et perspectives

Le projet atteint son objectif : un site e-commerce complet, testé et en ligne, enrichi d'une marketplace et d'une catégorie de prototypes réservée aux testeurs. Ce bilan retient les difficultés rencontrées, ce qu'elles m'ont appris et les évolutions possibles.

Difficultés rencontrées

Difficulté	Cause	Solution et enseignement
Erreur 500 sur la page des candidatures de testeurs	La migration avait été générée mais pas exécutée : la table n'existait pas en base	Exécuter `doctrine:migrations:migrate` ; contrôler l'état de la base après chaque migration
Fiche d'un produit non publié visible par un visiteur	Une comparaison `null` avec `null` donnait « propriétaire » à un produit sans vendeur	Comparer des identifiants et écrire un test qui couvre ce cas ; un test a permis de trouver le défaut

Difficulté	Cause	Solution et enseignement
Cache du navigateur absent en production	Le site est servi sous le chemin `/innovshop` : une règle `^/assets/` ne correspondait pas	Retirer l'ancrage de début de ligne dans `.htaccess` ; toujours tester dans l'environnement réel
Compression non constatée	Le test utilisait une requête HEAD, et le type JavaScript manquait à la liste	Tester avec une requête GET et compléter les types compressés
Outil de style inutilisable	La commande `php` du Mac pointait vers PHP 8.2, trop ancien pour l'outil	Passer par `symfony php` pour utiliser la bonne version
Prototype commandable par tout client	La règle n'existait pas : le catalogue ne distinguait pas les prototypes	Créer le rôle Testeur et une règle unique appliquée partout (vues, panier, commande)

Ce que j'en retiens

- Centraliser une règle (ici dans un Voter) évite d'oublier de la vérifier à un endroit.
- Un test écrit pour un cas limite révèle des défauts qu'un test manuel ne voit pas.
- La production se vérifie en production : chemins, versions de PHP et cache n'y sont pas ceux du poste local.

Évolutions possibles

- Paiement en ligne réel (le parcours de paiement par carte existe déjà, en mode simulé) et gestion des stocks.
- Avis et notes sur les produits.
- Pagination du catalogue.
- Validation d'un testeur directement depuis la fiche de candidature.
- Tableau de bord de statistiques pour l'administrateur.
- Corriger les points relevés par l'audit Lighthouse (description de la page, hiérarchie des titres, contrastes, poids du CSS et du JavaScript).

Annexes

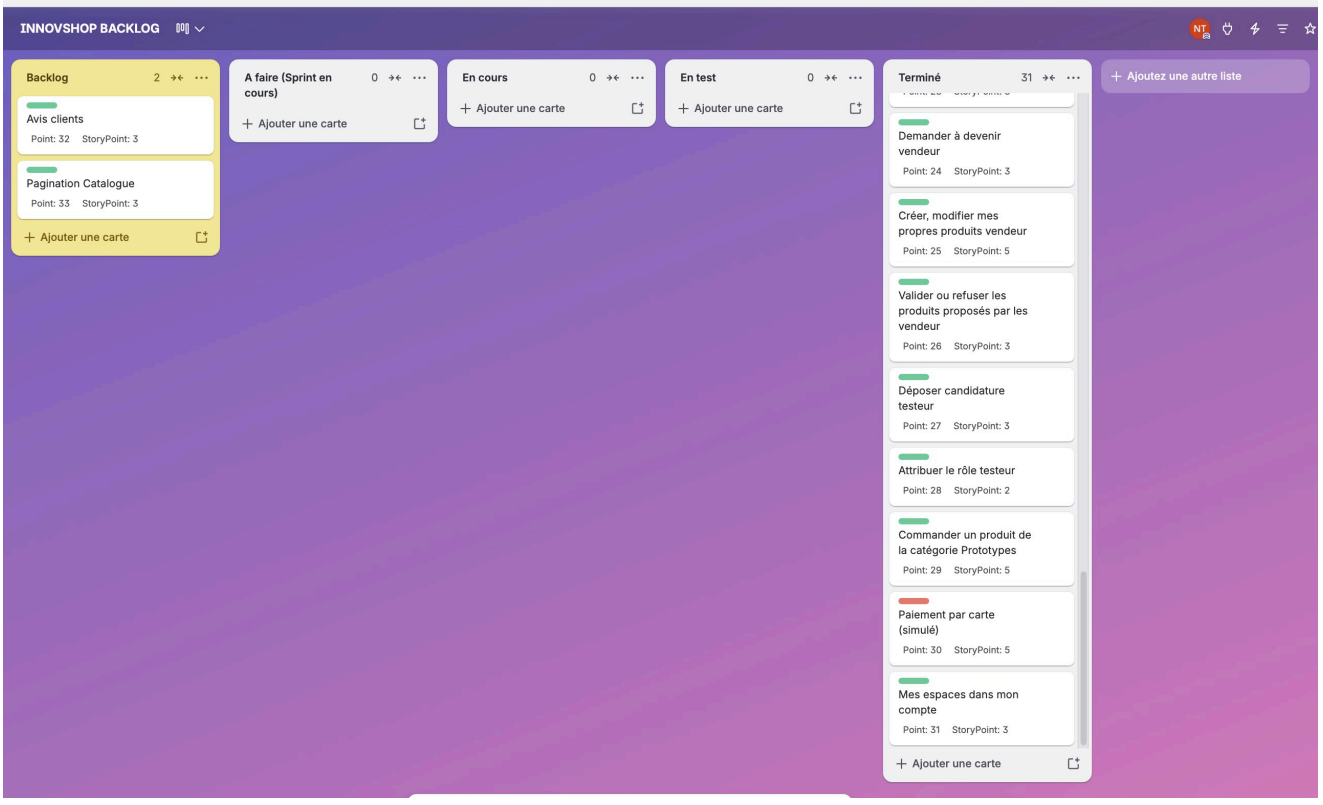
Annexe 1 : comptes de démonstration (installation locale)

Les fixtures créent quatre comptes qui partagent le mot de passe de démonstration `Demo-InnovShop-2026!`. Ils n'existent qu'en local.

Rôle	Adresse e-mail	Ce qu'on peut vérifier
Administrateur	admin@innovshop.local	Administration, modération des produits, attribution des rôles
Client	client@innovshop.local	Commande, historique ; le prototype propose « Devenir testeur »
Vendeur	vendeur@innovshop.local	Espace vendeur, produit en attente de validation
Testeur	testeur@innovshop.local	Peut commander un prototype

Annexe 2 : backlog détaillé

Le backlog compte 31 user stories : classées par priorité (méthode MoSCoW) et estimées en points de complexité. Le tableau Kanban public suit leur avancement : InnovShop Backlog sur Trello.



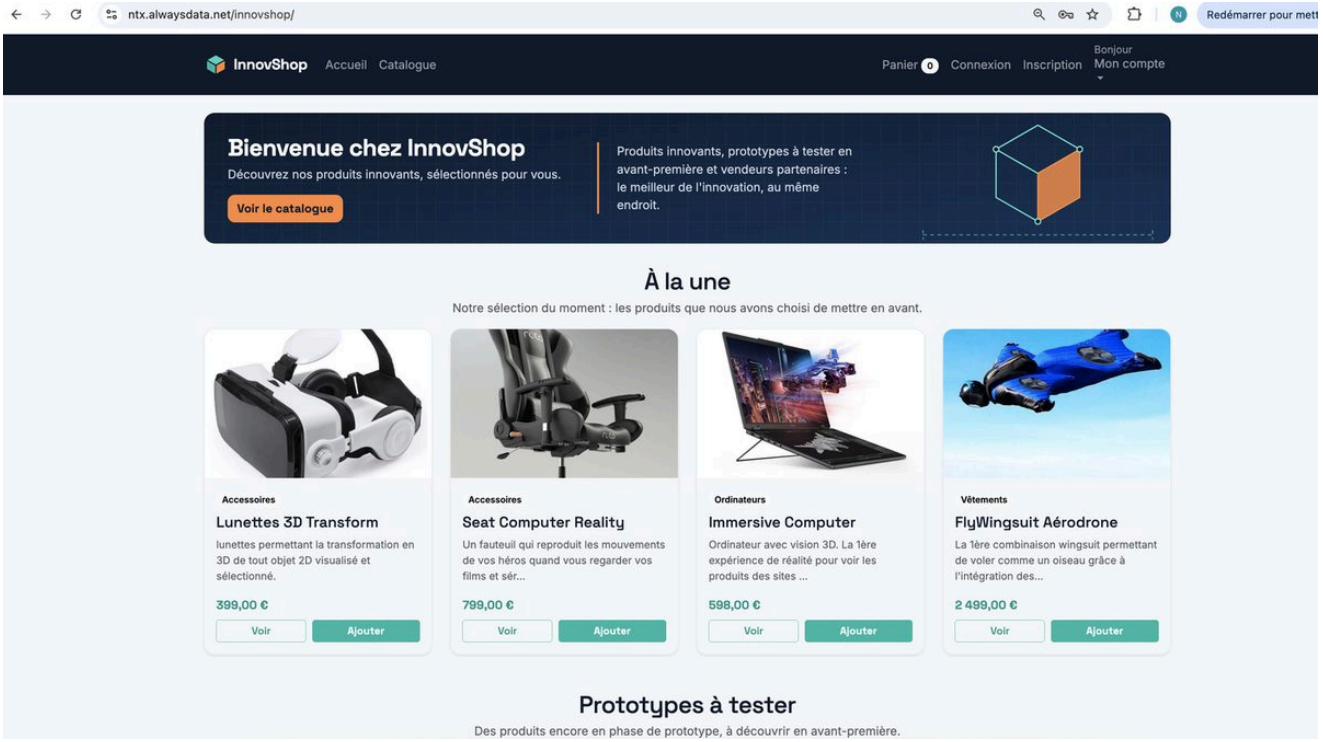
N°	Rôle	Fonctionnalité	Priorité	Points
1	Visiteur	Voir une page d'accueil graphique avec les derniers produits et les produits à la une	Must have	5

N°	Rôle	Fonctionnalité	Priorité	Points
2	Visiteur	Consulter le catalogue des produits	Must have	3
3	Visiteur	Consulter la fiche détaillée d'un produit	Must have	5
4	Visiteur	Ajouter un produit à mon panier	Must have	3
5	Visiteur	Supprimer un produit de mon panier	Must have	2
6	Visiteur	Consulter mon panier à tout moment	Must have	2
7	Visiteur	M'inscrire avec mon adresse e-mail et un mot de passe sécurisé	Must have	3
8	Client connecté	Me connecter à mon compte	Must have	3
9	Client connecté	Passer commande en plusieurs étapes	Must have	8
10	Client connecté	Recevoir un e-mail récapitulatif après commande	Must have	3
11	Client connecté	Consulter l'historique de mes commandes	Must have	3
12	Client connecté	Gérer mes informations de profil et de livraison	Should have	3
13	Visiteur	Être invité à me connecter pour passer commande	Must have	2
14	Administrateur	Ajouter un produit	Must have	3
15	Administrateur	Modifier un produit	Must have	3
16	Administrateur	Supprimer un produit	Must have	2
17	Administrateur	Visualiser toutes les commandes	Must have	3
18	Administrateur	Mettre à jour le statut d'une commande	Must have	3
19	Administrateur	Visualiser et gérer les comptes clients	Must have	2
20	Administrateur	Consulter l'historique des commandes d'un client	Should have	2
21	Client connecté	Modifier la quantité d'un produit dans le panier	Should have	3
22	Client connecté	Télécharger la facture PDF d'une commande	Should have	3
23	Client connecté	Recevoir un e-mail à chaque changement de statut	Should have	3
24	Client connecté	Demander à devenir vendeur	Should have	3

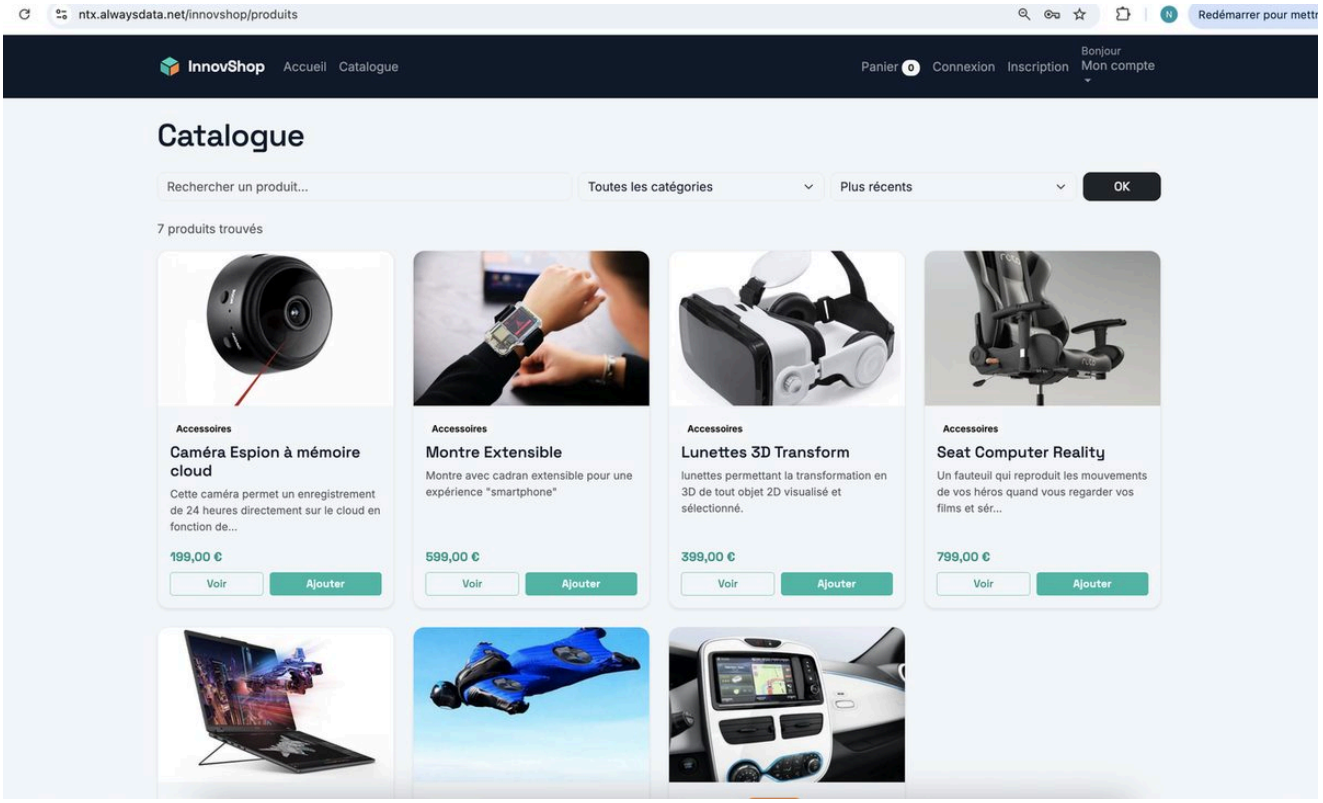
N°	Rôle	Fonctionnalité	Priorité	Points
25	Vendeur	Créer, modifier et supprimer mes produits	Should have	5
26	Administrateur	Valider ou refuser les produits des vendeurs	Should have	3
27	Visiteur	Déposer une candidature de testeur	Should have	3
28	Administrateur	Attribuer le rôle testeur	Should have	2
29	Testeur	Commander un produit de la catégorie « Prototypes »	Should have	5
30	Client connecté	Payer ma commande par carte bancaire	Must have	5
31	Client connecté	Retrouver mes espaces dans un menu « Mon compte »	Should have	3

Annexe 3 : captures d'écran du site en production

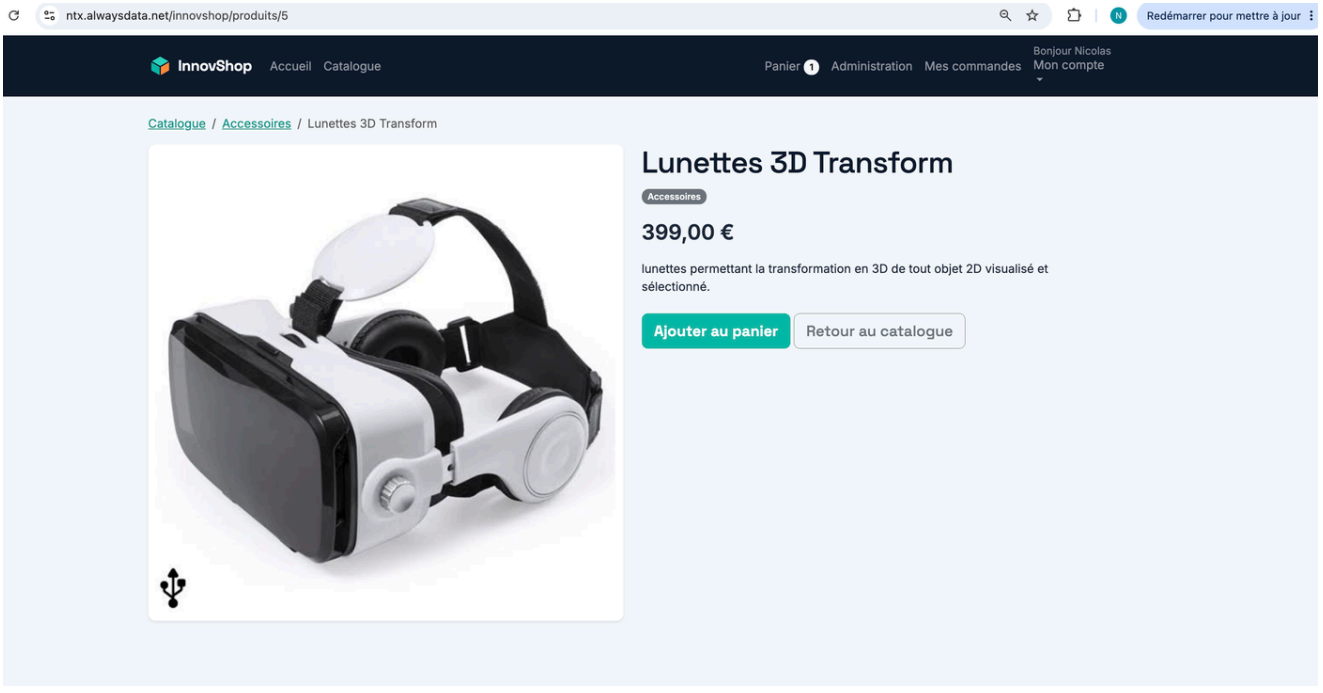
Accueil : bandeau de présentation, produits « À la une », prototypes à tester.



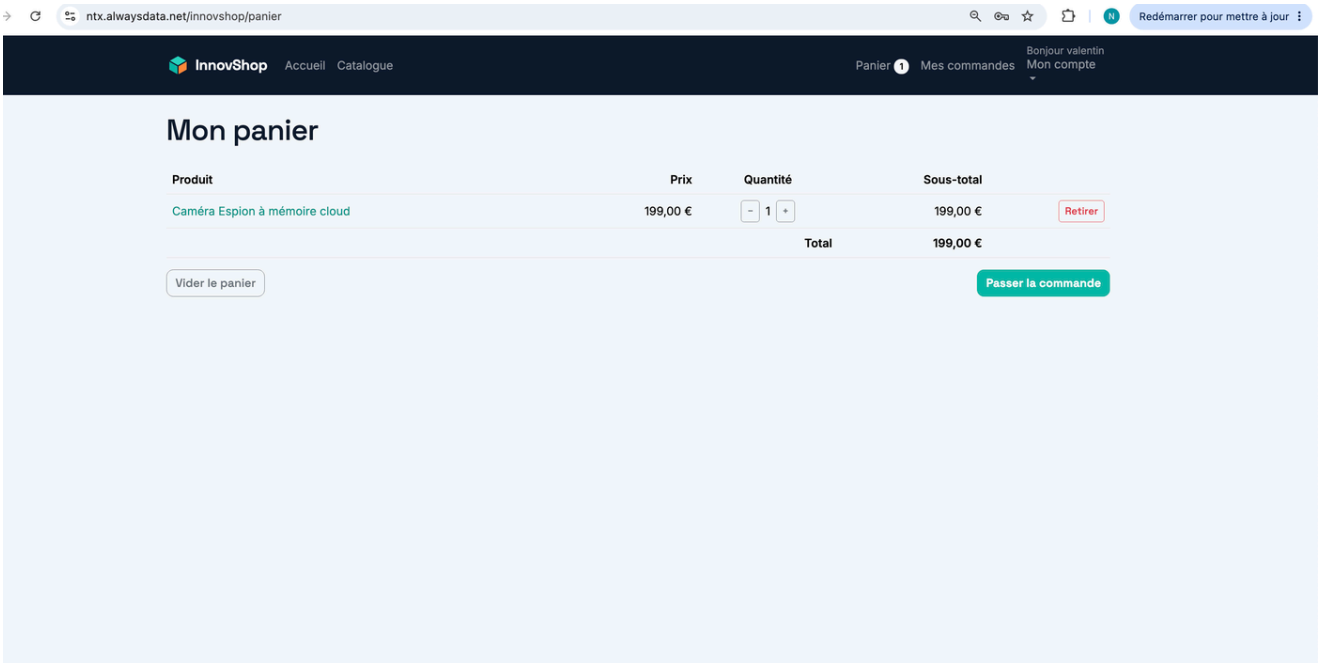
Catalogue : recherche, filtre par catégorie, tri, pastille « Prototype ».



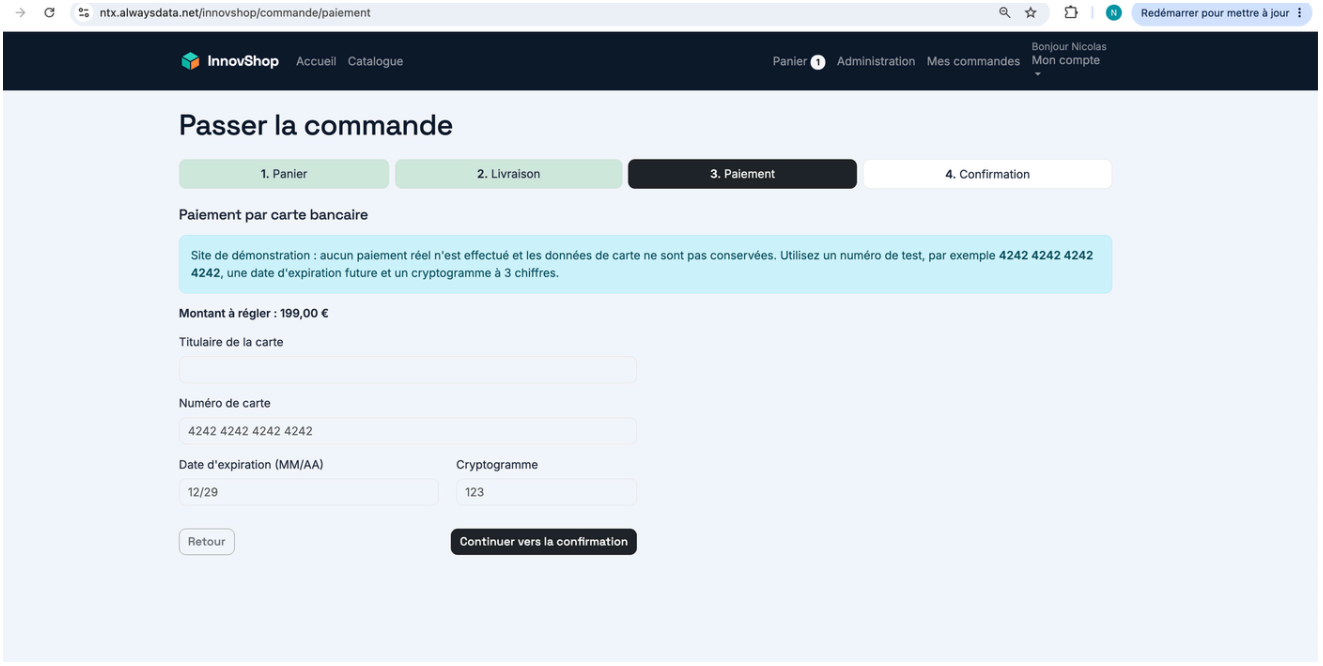
Fiche produit : description, options, ajout au panier.



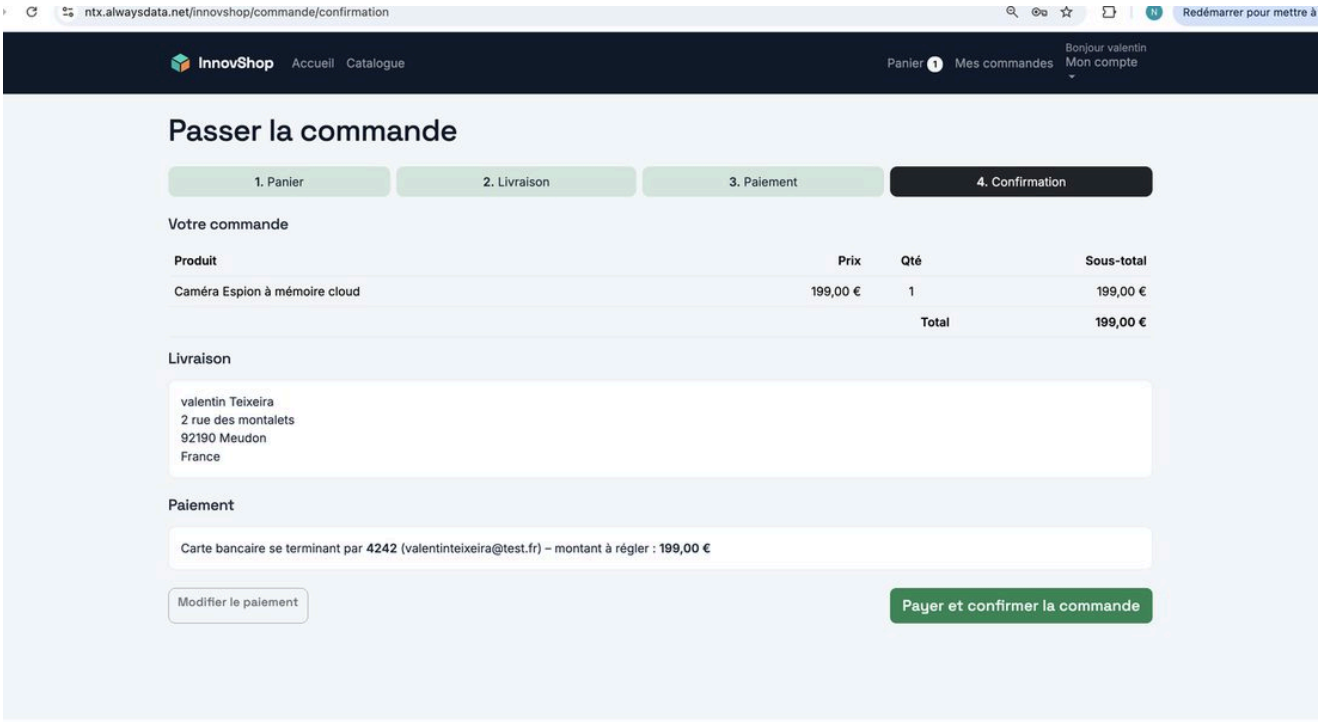
Panier : quantités modifiables, sous-totaux et total.

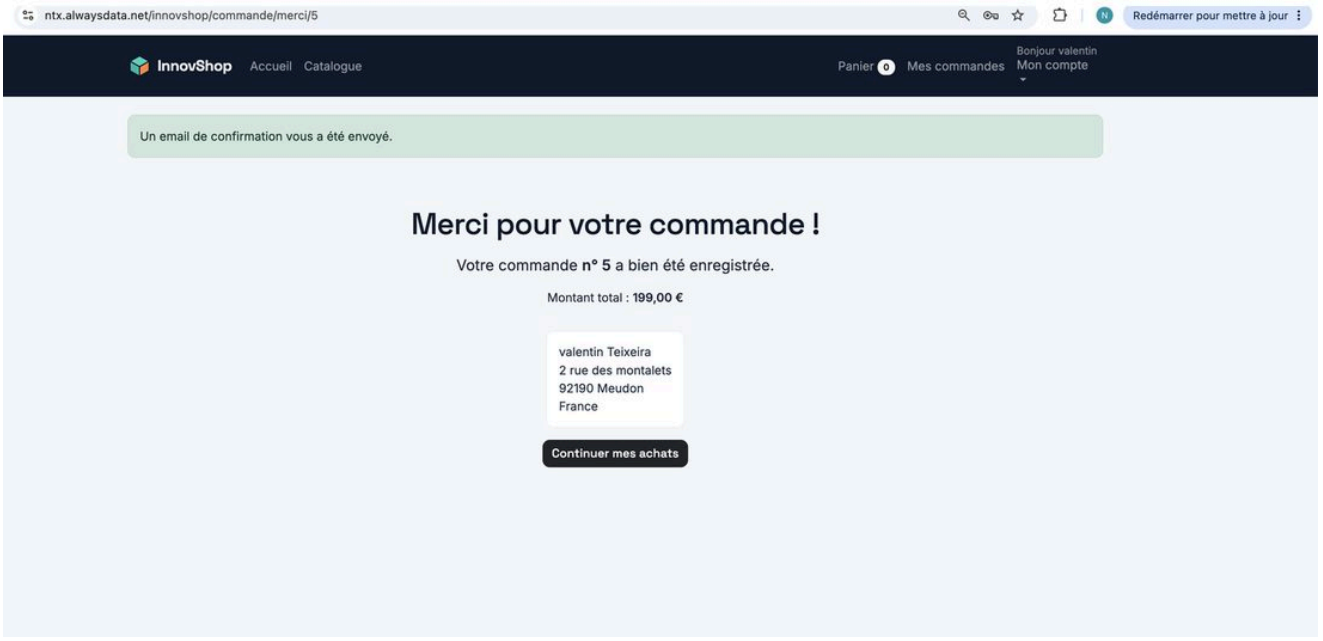


Paiement : formulaire de carte bancaire simulé (numéro, date d'expiration, cryptogramme) avec contrôle de la clé de Luhn ; aucune donnée de carte n'est conservée.

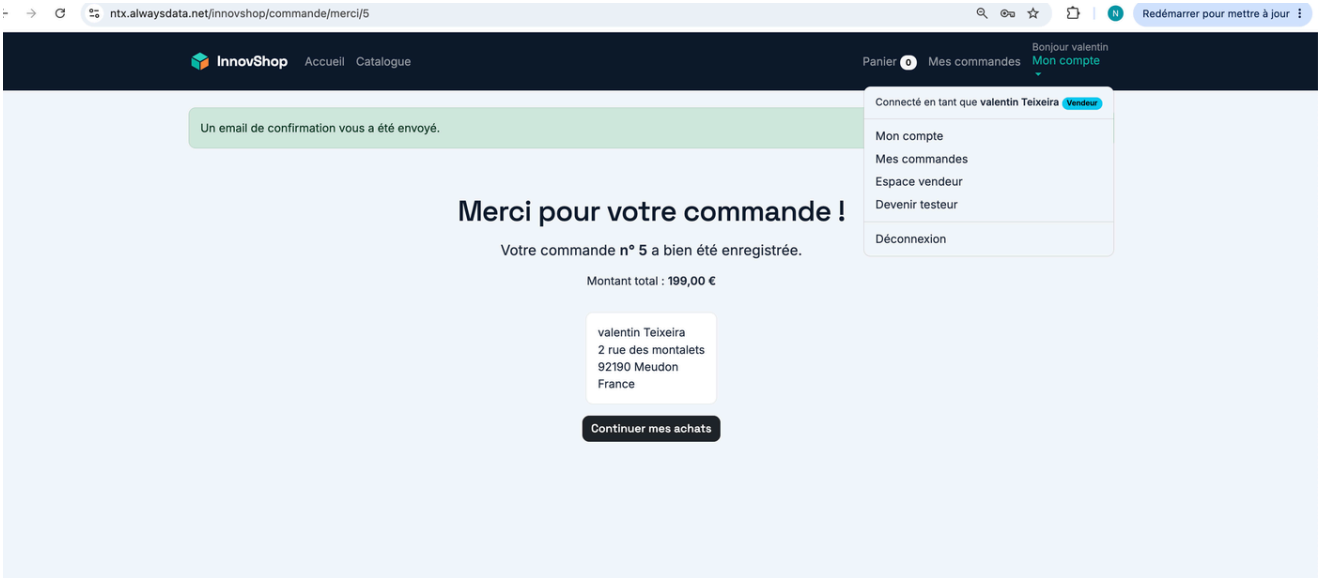


Confirmation et commande enregistrée : récapitulatif de la commande, de la livraison et du paiement avant validation, puis page de remerciement et e-mail de confirmation.

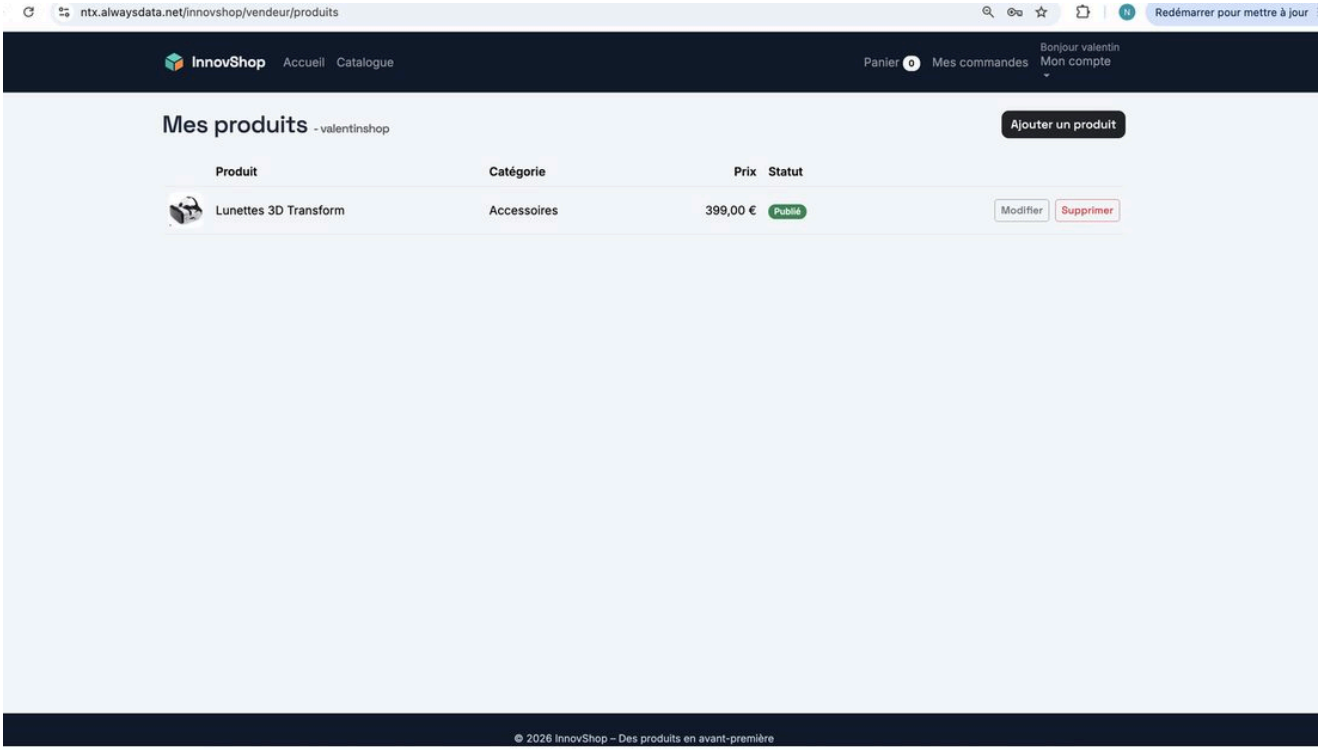


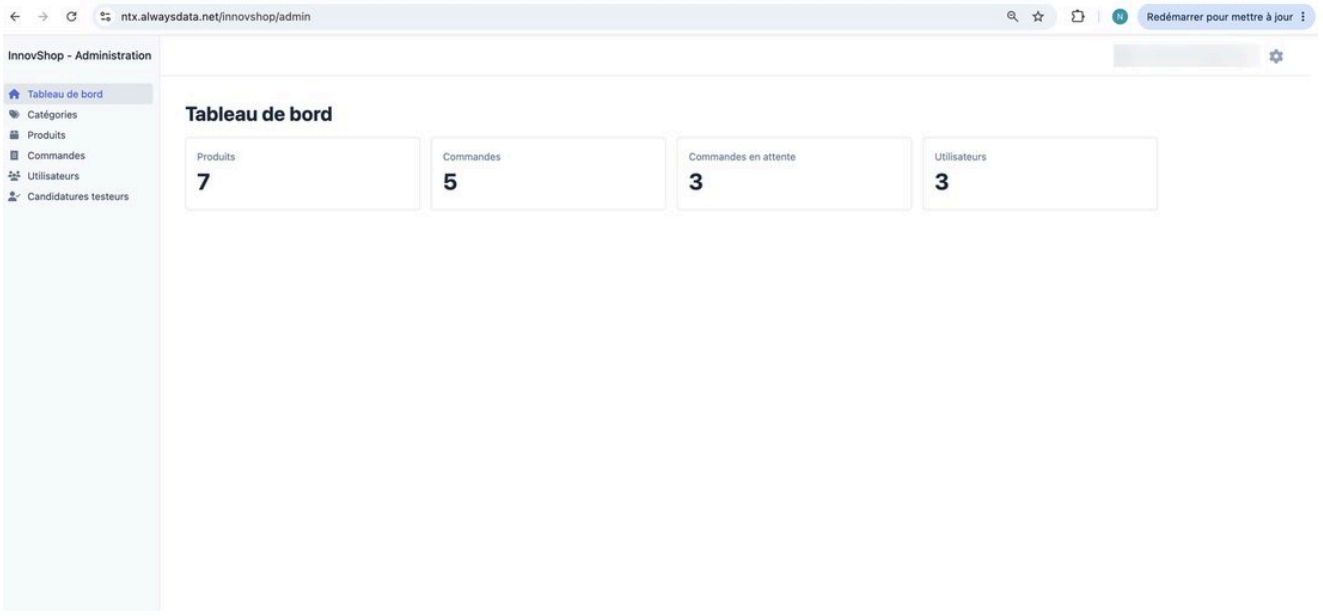


Menu « Mon compte » : commandes, espace vendeur, candidature testeur et déconnexion regroupés dans un menu déroulant.

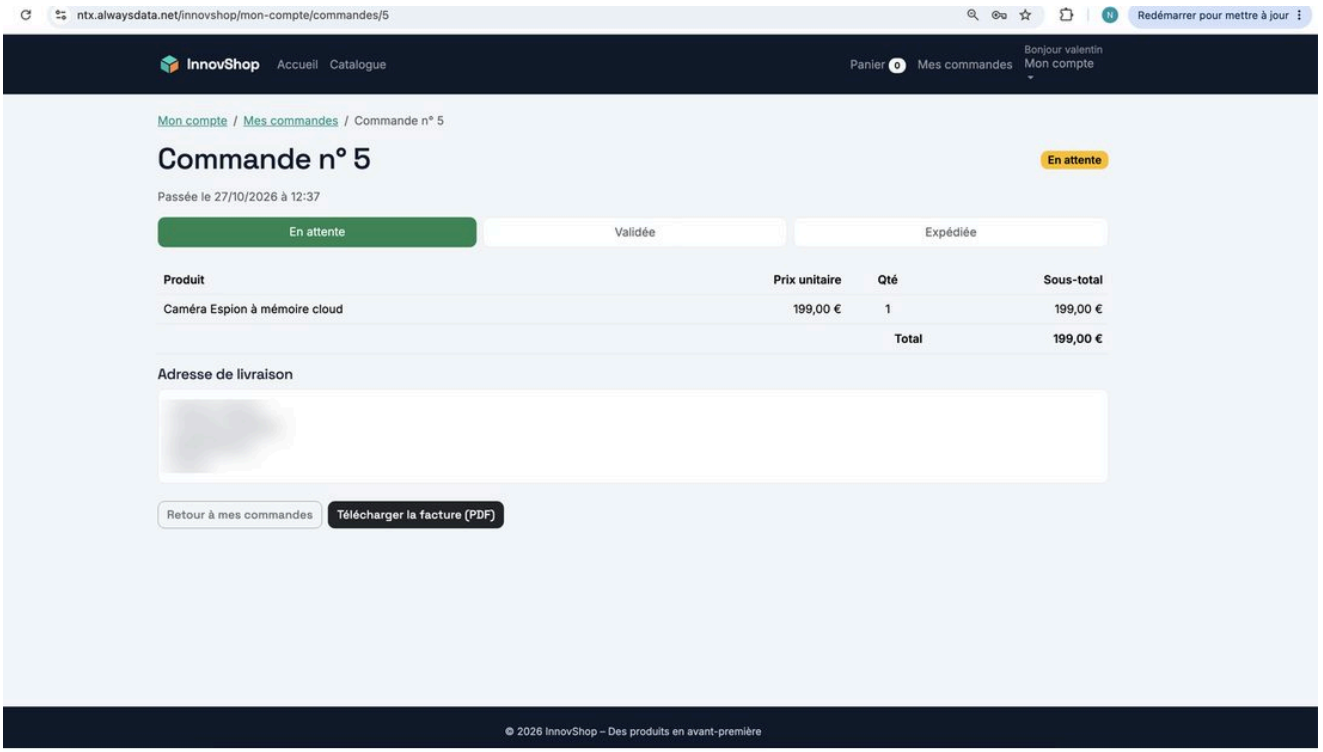


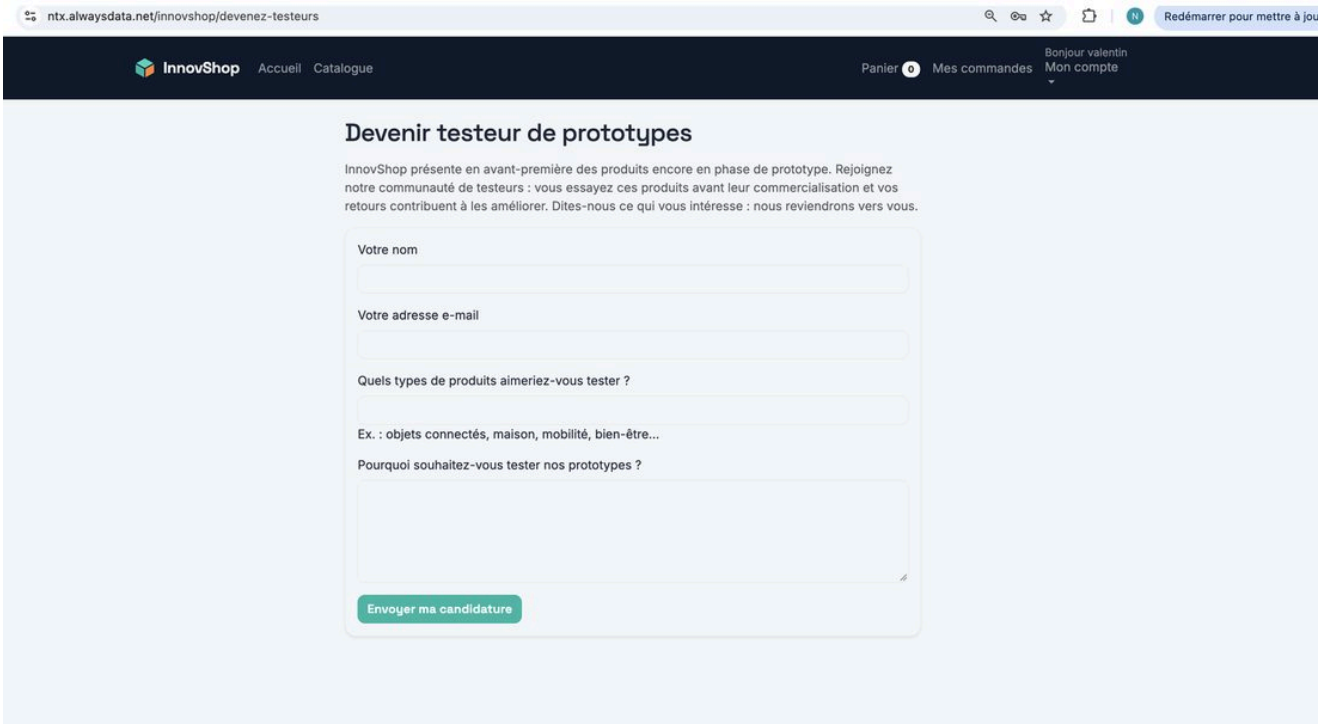
Espace vendeur et administration : liste des produits du vendeur avec leur statut de modération, puis tableau de bord EasyAdmin (adresse e-mail floutée).





Détail d'une commande et candidature testeur : suivi du statut, adresse de livraison (floutée) et facture PDF, puis formulaire pour devenir testeur de prototypes.





Annexe 4 : extraits de code

Cette annexe présente six extraits du code source, un par notion clé : controller Symfony, template Twig, JavaScript, règle d'accès (Voter), entité Doctrine et service métier. Les numéros de ligne sont ceux des fichiers du dépôt.

1. Controller Symfony : validation de la commande (`src/Controller/CheckoutController.php`)

CheckoutController.php src/Controller

```php
// Enregistrement de la commande
#[Route('/valider', name: 'app_checkout_valider', methods: ['POST'])]
public function valider(
    Request $request,
    CartService $cart,
    OrderService $orderService,
    CommandeMailer $commandeMailer,
): Response {
    if (!$this->isCsrfTokenValid('checkout', (string) $request->request->get('_token'))) {
        throw $this->createAccessDeniedException('Jeton CSRF invalide.');
    }

    if ($redirection = $this->redirigerSiPanierVide($cart)) {
        return $redirection;
    }

    $session = $request->getSession();
    $livraison = $session->get(self::SESSION_LIVRAISON);
    $utilisateur = $this->getUser();

    if (!$livraison) {
        return $this->redirectToRoute('app_checkout_livraison');
    }
    if (!$session->get(self::SESSION_PAIEMENT)) {
        return $this->redirectToRoute('app_checkout_paiement');
    }
    if (!$utilisateur instanceof Utilisateur) {
        throw $this->createAccessDeniedException();
    }

    $commande = $orderService->creerDepuisPanier($utilisateur, $livraison);
    $session->remove(self::SESSION_LIVRAISON);
    $session->remove(self::SESSION_PAIEMENT);
```

La route `/commande/valider` n'accepte que la méthode POST. Le controller vérifie le jeton CSRF, que le panier n'est pas vide, que les étapes livraison et paiement figurent bien en session et que l'utilisateur est connecté. Il confie ensuite la création de la commande au service `OrderService`, puis vide les données de session du tunnel.

2. Template Twig : bouton d'ajout au panier (`templates/produit/_card.html.twig`)

_card.html.twig templates/produit

```twig
{% if is_granted('PRODUIT_COMMANDER', produit) %}
    <form data-cart-form data-turbo="false" method="post" action="{{ path('app_cart_add', {id: produit.id}) }}" class="flex-fill d-grid">
        <input type="hidden" name="_token" value="{{ csrf_token('cart') }}">
        <button class="btn btn-sm btn-primary">Ajouter</button>
    </form>
{% else %}
    <a href="{{ path('app_testeur') }}" class="btn btn-sm btn-accent flex-fill"
       title="Ce prototype est réservé aux testeurs validés">Devenir testeur</a>
{% endif %}
```

La fonction `is_granted('PRODUIT_COMMANDER', produit)` interroge le Voter. Si l'accès est autorisé, la carte produit affiche un formulaire protégé par un jeton CSRF ; sinon elle propose le lien « Devenir testeur ». Le formulaire porte l'attribut `data-cart-form`, utilisé par le JavaScript.

3. JavaScript : ajout au panier sans rechargement (`assets/cart.js`)

cart.js assets

```
document.addEventListener('submit', async (event) => {
    const form = event.target.closest('form[data-cart-form]');
    if (!form) {
        return;
    }

    event.preventDefault();
    const bouton = form.querySelector('button');
    if (bouton) {
        bouton.disabled = true;
    }

    try {
        const reponse = await fetch(form.action, {
            method: 'POST',
            body: new FormData(form),
            headers: { 'X-Requested-With': 'XMLHttpRequest' },
        });
        if (!reponse.ok) {
            throw new Error('Réponse invalide');
        }
        const data = await reponse.json();
        majCompteur(data.count);
        notifier(data.message);
        await rafraichirPanier();
    } catch (erreur) {
        notifier('Une erreur est survenue, veuillez réessayer.', 'danger');
    } finally {
        if (bouton) {
            bouton.disabled = false;
        }
    }
});
```

Le script intercepte la soumission du formulaire (`preventDefault`), envoie les données avec `fetch` en POST (`FormData`), lit la réponse JSON, puis met à jour le compteur du panier, affiche une notification et rafraîchit le tableau du panier. Le bouton est désactivé pendant la requête, et une erreur affiche un message au visiteur.

4. Règle d'accès : le Voter des produits (`src/Security/Voter/ProduitVoter.php`)

ProduitVoter.php src/Security/Voter

```php
/**
 * Règles d'accès aux produits :
 * - un vendeur ne peut modifier ou supprimer que ses propres produits ;
 * - un prototype ne peut être commandé que par un testeur validé (ou un administrateur).
 */
class ProduitVoter extends Voter
{
    public const MODIFIER = 'PRODUIT_MODIFIER';
    public const COMMANDER = 'PRODUIT_COMMANDER';
…
    private function peutCommander(Produit $produit, TokenInterface $token): bool
    {
        // Les produits "normaux" sont ouverts à tous
        if (!$produit->isPrototype()) {
            return true;
        }

        if (!$token->getUser() instanceof Utilisateur) {
            return false;
        }

        return [] !== array_intersect(['ROLE_TESTEUR', 'ROLE_ADMIN'], $token->getRoleNames());
    }
}

```

Les produits classiques sont ouverts à tous. Un prototype n'est commandable que par un utilisateur connecté ayant le rôle testeur ou administrateur. La même règle est appelée depuis le template, le controller du panier et la validation de commande.

5. Entité Doctrine : le produit (`src/Entity/Produit.php`)

Produit.php src/Entity

```php
#[ORM\Entity(repositoryClass: ProduitRepository::class)]
#[ORM\Index(name: 'idx_produit_date_creation', columns: ['date_creation'])]
#[ORM\Index(name: 'idx_produit_statut', columns: ['statut'])]
…
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 150)]
    #[Assert\NotBlank(message: 'Le nom est obligatoire.')]
    #[Assert\Length(min: 2, max: 150)]
    private ?string $nom = null;

    #[ORM\Column(type: Types::TEXT)]
    #[Assert\NotBlank(message: 'La description est obligatoire.')]
    private ?string $description = null;

    #[ORM\Column(type: Types::DECIMAL, precision: 10, scale: 2)]
    #[Assert\NotBlank(message: 'Le prix est obligatoire.')]
    #[Assert\Positive(message: 'Le prix doit être supérieur à 0.')]
    private ?string $prix = null;
```

Les attributs PHP décrivent la table (colonnes, types, index) et les règles de validation (nom obligatoire, prix strictement positif). Doctrine en déduit la structure de la base et les migrations.

6. Service métier : création de la commande (`src/Service/OrderService.php`)

OrderService.php src/Service

```php
public function creerDepuisPanier(Utilisateur $utilisateur, array $livraison): Commande
{
    $items = $this->cartService->getItems();
    if ([] === $items) {
        throw new \LogicException('Impossible de commander : le panier est vide.');
    }

    $commande = new Commande();
    $commande
        ->setUtilisateur($utilisateur)
        ->setStatut(Commande::STATUT_EN_ATTENTE)
        ->setTotal($this->cartService->getTotal())
        ->setAdresseLivraison($this->formaterAdresse($utilisateur, $livraison));

    // Tout est enregistré dans une transaction : la commande et ses lignes, ou rien.
    $this->entityManager->wrapInTransaction(function () use ($commande, $items): void {
        $this->entityManager->persist($commande);

        foreach ($items as $item) {
            $produit = $item['produit'];

            $ligne = new LigneCommande();
            $ligne
                ->setNomProduit($produit->getNom())
                ->setPrixUnitaire($produit->getPrix())
                ->setQuantite($item['quantite'])
                ->setProduit($produit);

            $commande->addLigneCommande($ligne);
            $this->entityManager->persist($ligne);
        }
    });

    $this->cartService->clear();

    return $commande;
}
```

La commande et ses lignes sont enregistrées dans une seule transaction (`wrapInTransaction`) : tout est enregistré, ou rien. Chaque ligne conserve le nom et le prix du produit au moment de l'achat, même si le produit change ensuite.